

Singleton Design Pattern — Complete Guide (Beginner → Advanced, Python 3)

Every code example in this guide was actually run to confirm it behaves exactly as described — including a couple of nuances that most tutorials get wrong (see Section 8).

1. The Big Idea in One Line

A Singleton is a class that is allowed to create **only one object, ever** — and gives everyone in the program a way to reach that same object.

Think of the President of a country. There's exactly one President at a time. Every citizen who wants "the President" gets directed to the same person — nobody creates a second, personal President for themselves.

2. Formal Definition (the one interviewers expect)

┃ Ensure a class has only one instance, and provide a global point of access to it.

This comes from the original "Gang of Four" design patterns book. Two responsibilities live inside one class:

1. Do its normal job (whatever the class is for).
2. Control its own creation, so nobody can make a second copy.

Point 2 is what makes Singleton special — and, as you'll see in Section 12, also what makes some experienced engineers dislike it.

3. Why Do We Even Need This?

Picture a `Logger` class that writes messages to a log file. If every part of your app made its own `Logger` object:

- Every object opens its own file handle to the same file — wasted memory, possible file-locking conflicts.
- Log lines might get written out of order or overwrite each other.
- You'd have to manually pass "the one true logger" around everywhere instead of just asking for it.

Other classic real-world candidates: database connection pools, configuration/settings managers, thread pools, caches, and single-hardware access (like a printer spooler).

The pattern's job: guarantee "one and only one" plus a simple, global way to fetch it.

4. The Structure

(This is the diagram shown above in chat.)

- **Singleton class:** holds a private class-level slot that stores the one instance, plus a controlled way to create it the first time and hand back the same one every other time.
- **Client:** anyone using the Singleton. The client never creates the object with a plain constructor call in the classic (Java-style) version — it always asks through a controlled access point.

In Python this gets both simpler and stranger, because Python doesn't let you make a constructor truly private, and because Python already gives you a built-in singleton for free — **modules** (Section 6.4).

5. The Most Basic Implementation (Naive Version)

```
python

class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

s1 = Singleton()
s2 = Singleton()

print(s1 is s2)          # True - same object in memory
print(id(s1) == id(s2))  # True
```

Why `__new__` and not `__init__`?

- `__new__` is the method that actually *creates* the object — it runs before `__init__`.
- `__init__` only *initializes* an object that already exists.
- To control whether a new object gets created at all, you have to hook into `__new__`.

⚠ **Gotcha #1:** `__init__` runs every single time

Even though `__new__` correctly returns the *same* object on the second call, Python **always** calls `__init__` afterward — whether the object was freshly created or reused.

```
python
```

```

class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self):
        print("Running __init__...")  # prints EVERY time you call Singleton()

Singleton()  # Running __init__...
Singleton()  # Running __init__...  <- ran again!

```

If `__init__` does real work (opening a connection, loading a config file), that work re-runs every time. Fix it with a guard flag:

```

python

class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self):
        if getattr(self, "_initialized", False):
            return
        self._initialized = True
        print("Running real setup only once")

```

Keep this one in your back pocket — interviewers love asking "does `__init__` run twice here?"

6. Seven Ways to Build a Singleton in Python

Python doesn't force one "correct" way to do this. Here are the ones you should actually know, roughly simplest → most robust.

6.1 `__new__` override (what you just saw)

Already covered above — simple, explicit, the classic OOP-interview answer. It has a real bug with inheritance:

python

```
class Singleton:
    _instance = None
    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

class LoggerSub(Singleton):
    pass

a = Singleton()
b = LoggerSub()
print(type(b)) # <class '__main__.Singleton'> <- BUG! You wanted a LoggerSub.
```

Why: `LoggerSub` doesn't define its own `_instance`, so `cls._instance` looks it up on the parent `Singleton` — and since `Singleton()` was called first, that slot is already filled, so `LoggerSub()` just hands back the *parent's* object.

The fix — check whether `_instance` is defined directly on *this* class, not inherited:

python

```
class Singleton:
    def __new__(cls, *args, **kwargs):
        if "_instance" not in cls.__dict__:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Now every subclass correctly gets its own instance. (Verified: `type(LoggerSub())` is now correctly `LoggerSub`.)

6.2 Decorator-based Singleton

python

```
def singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@singleton
class DatabaseConnection:
    def __init__(self):
        print("Opening database connection...")

db1 = DatabaseConnection()
db2 = DatabaseConnection()
print(db1 is db2)  # True
```

Clean and short, but it has a real downside — a favorite interview trick question:

```
python

isinstance(db1, DatabaseConnection)
# TypeError: isinstance() arg 2 must be a type...
# DatabaseConnection is no longer a class, it's a function (get_instance)!
```

You also can't subclass `DatabaseConnection` anymore — it isn't a class. And (verified) trying to `pickle.dumps(db1)` fails outright with a `PicklingError`, since pickle can't resolve the name `DatabaseConnection` back to a class in the module.

6.3 Metaclass-based Singleton (the "senior engineer" answer)

A metaclass is "a class whose instances are classes." Overriding `__call__` on the metaclass lets you intercept every `SomeClass()` call.

```
python
```

```

class SingletonMeta(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]

class Logger(metaclass=SingletonMeta):
    def __init__(self):
        print("Logger initialized")

l1 = Logger()
l2 = Logger()
print(l1 is l2)  # True

```

Why this is usually the strongest OOP answer in interviews:

- `isinstance()` still works normally — `Logger` is still a real class.
- Inheritance works correctly *automatically*: `_instances` is keyed by the exact `cls`, so a subclass naturally gets its own slot — no manual `__dict__` check needed like in 6.1. (Verified: a `SubLogger(Logger)` gets its own instance, distinct from `Logger`'s.)
- This is the same mechanism real frameworks use internally (ORMs, Django's model machinery), so it signals you understand Python's object model.

6.4 Just Use a Module (the "practical Python engineer" answer)

Every Python module is already a singleton. When you `import config`, Python checks `sys.modules` first — if `config` was already imported anywhere in your program, you get back the *exact same* module object; the file is not re-executed.

```

python

# config.py
connection_string = "postgres://localhost/mydb"
debug_mode = True

```

```
python
```

```
# main.py
import config
config.debug_mode = False

# elsewhere.py
import config
print(config.debug_mode) # False – same module object, shared state
```

No metaclasses, no `__new__` tricks — just plain module-level variables. Many experienced Python engineers consider this the actual go-to for config and constants, and dropping it in an interview shows real-world judgment, not just pattern memorization.

6.5 Borg / Monostate Pattern (share the *state*, not the *identity*)

A clever variant popularized by Alex Martelli. Instead of forcing one object to exist, it lets you create as many objects as you want — but they all secretly share the same internal `__dict__`, so they behave identically.

```
python

class Borg:
    _shared_state = {}

    def __new__(cls, *args, **kwargs):
        obj = super().__new__(cls)
        obj.__dict__ = cls._shared_state
        return obj

b1 = Borg()
b2 = Borg()

b1.x = 5
print(b2.x) # 5 – state is shared
print(b1 is b2) # False! Different objects...
print(b1.__dict__ is b2.__dict__) # True – ...but same underlying state
```

Interview-worthy distinction: **Singleton controls identity** (`is`); **Borg controls state**. Borg plays more nicely with subclassing and side-steps some of the identity-related headaches from Section 8.

6.6 Thread-Safe Singleton (Double-Checked Locking)

The naive version has a real race condition. CPython's GIL means only one thread runs Python bytecode at a time — but a single line like `if cls._instance is None:` compiles into *multiple* bytecode instructions, and the OS can switch threads between any of them.

Two threads can both see `_instance is None` at the same moment and both try to create an object.

```
python

import threading

class ThreadSafeSingleton:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:           # 1st check – fast, no lock
            with cls._lock:                 # pay the lock cost once
                if cls._instance is None:   # 2nd check – inside the lock
                    cls._instance = super().__new__(cls)
        return cls._instance
```

(Verified with 20 concurrent threads all racing to call this — every one of them ends up with the exact same instance.)

Why check twice? This is **double-checked locking**, a classic interview question:

- The **first check** (no lock) is a fast path — once the instance exists (99.9% of calls), you skip the expensive lock entirely.
- The **second check** (inside the lock) protects against the race: if two threads both got past the first check before either created the instance, only one gets the lock first, creates the object, and releases it. The second thread then enters the lock, re-checks, sees the instance now exists, and skips creating a duplicate.

Only one check outside the lock → fast but not thread-safe. Only one check inside the lock → correct but every call pays the locking cost forever. Double-checking gives you both.

6.7 Enum-based Singleton

```
python
```



```

from enum import Enum

class AppConfig(Enum):
    INSTANCE = "config"

    def get_setting(self, key):
        return {"debug": True}.get(key)

a = AppConfig.INSTANCE
b = AppConfig.INSTANCE
print(a is b)  # True

```

Python builds enum members exactly once at import time, so this is automatically thread-safe, and (verified) automatically safe against the pickling/copying issues in Section 8 — Python has dedicated built-in support for pickling enum members by name. The trade-off: it looks unusual to most engineers, doesn't support lazy initialization (the instance is built at import time, not on first use), and doesn't support normal subclassing.

7. Which One Should You Actually Pick?

Approach	Thread-safe out of the box?	Lazy (created on first use)?	Inheritance-safe?	<code>isinstance()</code> works?
<code>__new__</code> (naive)	No	Yes	No — needs the <code>cls.__dict__</code> fix	Yes
<code>__new__</code> (fixed)	No	Yes	Yes	Yes
Decorator	No	Yes	Breaks — can't subclass	No — breaks!
Metaclass	No	Yes	Yes, automatically	Yes
Module	Yes	Eager (at import)	N/A	N/A
Borg	Mostly	Yes	Yes	Yes (but <code>is</code> fails)
Thread-safe lock	Yes	Yes	Yes (with the <code>cls.__dict__</code> fix)	Yes
Enum	Yes	Eager	Very limited	Yes

In an interview, if asked "how would you implement Singleton in Python?" — lead with the **metaclass** version as your main answer (the most robust general-purpose OOP solution),

mention that in real production Python code you'd often just reach for a **module**, and bring up the **thread-safe** / **double-checked locking** version if multithreading comes up. Covering all three shows range.

8. Pickling and Copying — A Subtlety Most Tutorials Get Wrong

Most tutorials just say "pickling breaks singletons, add `__reduce__`." That's actually **false** for one of the most common implementations. Here's the real mechanism, verified by actually running it.

Both `pickle` and the `copy` module (`copy.copy`, `copy.deepcopy`) use the same underlying protocol: `__reduce_ex__`. Unless you override it, Python's default behavior reconstructs your object by calling `cls.__new__(cls)` and then restoring its `__dict__`. That one fact explains everything below.

`__new__`-based singleton (Section 6.1) — uniqueness is enforced *inside* `__new__` itself. When pickle/copy calls `cls.__new__(cls)` to rebuild the object, it runs straight through your existing check and returns the cached instance. Identity survives automatically, with zero extra code:

```
python

import pickle, copy
s1 = Singleton()
s2 = pickle.loads(pickle.dumps(s1))
s3 = copy.deepcopy(s1)
print(s1 is s2, s1 is s3)  # True True — nobody wrote a single line of defense
```

Metaclass-based singleton (Section 6.3) — uniqueness is enforced in the metaclass's `__call__`, not in `__new__`. Default reconstruction calls `cls.__new__(cls)` *directly*, completely bypassing the metaclass's `__call__` (and the dictionary that tracks the singleton) — so you silently get a second, untracked object:

```
python

m1 = Logger()
m2 = pickle.loads(pickle.dumps(m1))
m3 = copy.deepcopy(m1)
print(m1 is m2, m1 is m3)  # False False — genuinely broken
```

The fix — one method fixes pickling, `copy.copy`, and `copy.deepcopy` all at once, because all three defer to it:

```
python
```

```
class Logger(metaclass=SingletonMeta):
    def __init__(self):
        ...
    def __reduce__(self):
        return (self.__class__, ())
```

This tells Python "to rebuild me, just call `Logger()` again," which correctly routes back through `SingletonMeta.__call__` and returns the cached instance. (Verified: with this fix, pickling, `copy.copy`, and `copy.deepcopy` all correctly return the same object.)

Decorator-based singleton (Section 6.2) — doesn't even reach the "broken identity" stage; pickling fails outright with a `PicklingError`, since the name `DatabaseConnection` in the module now refers to the wrapper function, not a class.

Enum-based singleton (Section 6.7) — pickling and copying both correctly preserve identity automatically, no extra code needed.

Interview-ready takeaway: don't just say "pickling breaks singletons, add `__reduce__`" — say *it depends on where your class enforces uniqueness*. If it's inside `__new__`, you get pickle/copy safety for free. If it's anywhere else (a metaclass, a decorator), you need to explicitly redirect reconstruction back through your normal creation path with `__reduce__`.

9. The Multiprocessing Gotcha (almost nobody mentions this)

If you use Python's `multiprocessing` module, each child process gets its **own separate Python interpreter and its own memory space**. Your "singleton" is only a singleton *within one process*. Spin up several worker processes and you get one separate "single" instance per process — none of them share state.

Verified directly: the main process's singleton `id()` and each child process's singleton `id()` all came out different. If you need one object truly shared across processes, a Python-level singleton class cannot do it — you need real inter-process tools (`multiprocessing.Manager`, a database row, Redis, etc.). This is a great gotcha to raise unprompted in an interview; it signals depth beyond textbook knowledge.

10. Singleton vs. Static Class vs. Global Variable

People often blur these three together:

- **Global variable** — just a name at module level holding an object. No enforcement; anyone can create another one. No lazy initialization, no lifecycle control.
- **Static class** (only classmethods) — no object is ever created at all; you just call functions namespaced under a class. There's no "instance" to speak of.
- **Singleton** — a real object is created (so it can hold state, support inheritance, be passed around as `self`), but creation is restricted to exactly once.

Interview-ready one-liner: "A *global variable has no protection against reassignment or duplication; a static class has no state-carrying instance at all; a Singleton gives you a real, stateful object while still guaranteeing there's only ever one.*"

11. Real-World Examples You Can Quote

- `logging.getLogger(name)` in the standard library — calling it twice with the same name returns the same logger object.
- Database connection pools, created once and reused across the app.
- Configuration/settings objects loaded once at startup.
- Caching layers — one shared cache instead of several independent caches stepping on each other.
- Thread pools / process pools — one pool coordinating a fixed set of workers, not several competing pools.

12. Why Some Engineers Call Singleton an "Anti-Pattern"

A very common follow-up: "What are the downsides of Singleton?"

1. **Hidden global state** — any code anywhere can silently depend on and mutate it. Much harder to trace than an explicitly passed-in dependency.
2. **Violates the Single Responsibility Principle** — the class does its actual job *and* manages its own lifecycle/creation.
3. **Hard to unit test** — since the instance is shared and often persists for the whole test run, state can leak between unrelated tests, causing flaky, order-dependent failures.
4. **Tight coupling** — code that calls `Singleton.get_instance()` directly is coupled to that concrete class; you can't easily swap in a mock, unlike with dependency injection.
5. **Hides dependencies** — a function that secretly reaches for a global singleton *looks* like it needs no inputs, but actually has a hidden one.

When it's still the right call: logging, low-level resource management, or genuinely process-wide config — cases where "there really is only one of these in the real world," not just "I don't feel like passing this argument around."

13. Testing Code That Depends on a Singleton

Common follow-up: "Your code calls `Logger.get_instance()`. How do you unit test it?"

- **Reset the instance between tests** (e.g. a `tearDown` that sets `Logger._instance = None`) — simple but a bit hacky.
- **Dependency injection instead of a hard-coded singleton call** — pass the logger into whatever needs it, and let a singleton be *one possible* way to obtain that object at the very top of your program, not something buried deep in business logic.
- **Mock/patch** the singleton's access method in tests using `unittest.mock.patch`.

Mentioning dependency injection as the deeper fix (not just a patch) reads as a stronger, more senior answer.

14. Interview Question Bank

Q1: What problem does the Singleton pattern solve? It guarantees a class has exactly one instance and gives a single, well-known way to access it — useful when more than one instance would cause bugs or wasted resources.

Q2: How do you implement Singleton in Python? Cover 2–3 approaches: `__new__` override, metaclass, and mention modules are singletons by default. Lead with the metaclass version as the "real" OOP implementation.

Q3: Is your Singleton thread-safe? How do you make it thread-safe? The naive version isn't. Explain double-checked locking with `threading.Lock` (Section 6.6).

Q4: Does the GIL make Singleton creation automatically thread-safe? No. The GIL guarantees only one thread runs Python bytecode at a time, but a compound check-then-act statement like `if cls._instance is None: cls._instance = ...` is multiple bytecode instructions — a thread switch can happen between them, so the race condition is still real.

Q5: What happens if you pickle and unpickle a Singleton? It depends on where uniqueness is enforced. With the `__new__`-override version, identity is preserved automatically for free. With a metaclass-based version, pickling *breaks* identity by default, because default unpickling calls `cls.__new__(cls)` directly and skips the metaclass's `__call__` — you need `__reduce__` to fix that specifically (Section 8).

Q6: Is a Singleton still a singleton across multiple processes? No — each process has its own memory space and its own copy of the "singleton." True cross-process sharing needs external tools like Redis, a database, or `multiprocessing.Manager` (Section 9).

Q7: What's the difference between Singleton and a global variable? A global variable has zero protection against reassignment or duplication; a Singleton actively enforces "only one" through its creation logic.

Q8: What's the difference between Singleton and a static/utility class? A static class never creates an instance at all — just namespaced functions. A Singleton creates one real, stateful object.

Q9: What's the Borg/Monostate pattern, and how does it differ from Singleton? Borg allows multiple objects to exist, but they all share the same internal state (`__dict__`), so `is` comparisons fail but behavior is identical. Singleton controls identity; Borg controls shared state.

Q10: Why is Singleton sometimes called an anti-pattern? Global hidden state, SRP violation, hard to unit test, tight coupling, hides dependencies (Section 12).

Q11: How would you unit test code that depends on a Singleton? Reset the instance in test teardown, or better, refactor toward dependency injection so the singleton is created once

at the top of the app and passed down explicitly.

Q12: Does `__init__` run every time you call `Singleton()`? Yes — always, even when `__new__` returns a cached object. Guard it manually with a flag if you don't want repeated initialization.

Q13: What breaks if you use a decorator to implement Singleton? `isinstance()` checks fail (the class name now refers to a function), you can't subclass it, and pickling fails outright with a `PicklingError`.

Q14: How does the metaclass approach handle subclassing better than the raw `__new__` approach? `__instances__` is a dictionary keyed by the exact class, so each subclass automatically gets its own entry — no manual fix needed, unlike the `__new__` version where subclasses need an explicit `cls.__dict__` check.

Q15: Why are Python modules considered "free" singletons? `sys.modules` caches every module after its first import — every subsequent `import` of the same module returns the exact same object without re-running the file.

Q16: Name a real singleton-like object in the Python standard library.

`logging.getLogger(name)` — repeated calls with the same name return the same logger instance.

Q17: What SOLID principle does Singleton most directly conflict with? Single Responsibility Principle — the class manages both its business logic and its own instantiation lifecycle.

Q18: Eager vs. lazy Singleton initialization — what's the trade-off? Eager (created immediately, e.g. at import time) is simpler and naturally thread-safe, but wastes resources if never used. Lazy (created on first access) saves resources but needs explicit thread-safety handling.

15. One-Page Cheat Sheet

- **Definition:** one instance, one global access point.
- **Simplest Python answer:** modules are singletons already — `import` caches by default.
- **Classic OOP answer:** metaclass with `__call__` override, keyed by `cls`.
- **Thread safety:** double-checked locking with `threading.Lock`.
- **Bug #1:** `__init__` runs every call — guard it with a flag.
- **Bug #2:** naive `__new__` breaks with subclasses — check `"_instance" not in cls.__dict__`, not just `cls._instance`.
- **Bug #3:** decorator-based version breaks `isinstance()`, subclassing, and pickling entirely.
- **Pickling/copying:** safe for free with `__new__`-based singletons; broken by default for metaclass-based ones — fix with a single `__reduce__` method.

- **Multiprocessing:** singletons never cross process boundaries — each process gets its own.
 - **Criticism:** hidden global state, SRP violation, hard to test.
 - **Good real uses:** logging, config, connection pools, caches.
 - **Bad use:** reaching for Singleton just to avoid passing an argument around — usually better solved with dependency injection.
-

This pairs naturally with your broader LLD prep — a Cache Manager wrapping an LRU Cache, for example, is a very natural place to apply the metaclass-based Singleton, since you want exactly one cache instance shared across the whole application.